

1. 3D Object View

```
#include <GL/glut.h>
#include <iostream>
using namespace std;

float X1, Y1, Z1;
float X2, Y2, Z2;

void drawCubeFromPoints(float x1, float y1, float z1,
                        float x2, float y2, float z2)
{
    glBegin(GL_QUADS);

    // Front face
    glColor3f(1, 0, 0);
    glVertex3f(x1, y1, z1);
    glVertex3f(x2, y1, z1);
    glVertex3f(x2, y2, z1);
    glVertex3f(x1, y2, z1);

    // Back face
    glColor3f(0, 1, 0);
    glVertex3f(x1, y1, z2);
    glVertex3f(x1, y2, z2);
    glVertex3f(x2, y2, z2);
    glVertex3f(x2, y1, z2);

    // Left face
    glColor3f(0, 0, 1);
    glVertex3f(x1, y1, z1);
    glVertex3f(x1, y1, z2);
```

```
glVertex3f(x1, y2, z2);
glVertex3f(x1, y2, z1);

// Right face
glColor3f(1, 1, 0);
glVertex3f(x2, y1, z1);
glVertex3f(x2, y1, z2);
glVertex3f(x2, y2, z2);
glVertex3f(x2, y2, z1);

// Top face
glColor3f(0, 1, 1);
glVertex3f(x1, y2, z1);
glVertex3f(x1, y2, z2);
glVertex3f(x2, y2, z2);
glVertex3f(x2, y2, z1);

// Bottom face
glColor3f(1, 0, 1);
glVertex3f(x1, y1, z1);
glVertex3f(x2, y1, z1);
glVertex3f(x2, y1, z2);
glVertex3f(x1, y1, z2);

glEnd();
}

void display()
{
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();
    gluLookAt(15, 15, 25,
```

```
        0, 0, 0,
        0, 1, 0);

drawCubeFromPoints(X1, Y1, Z1, X2, Y2, Z2);
glutSwapBuffers();
}

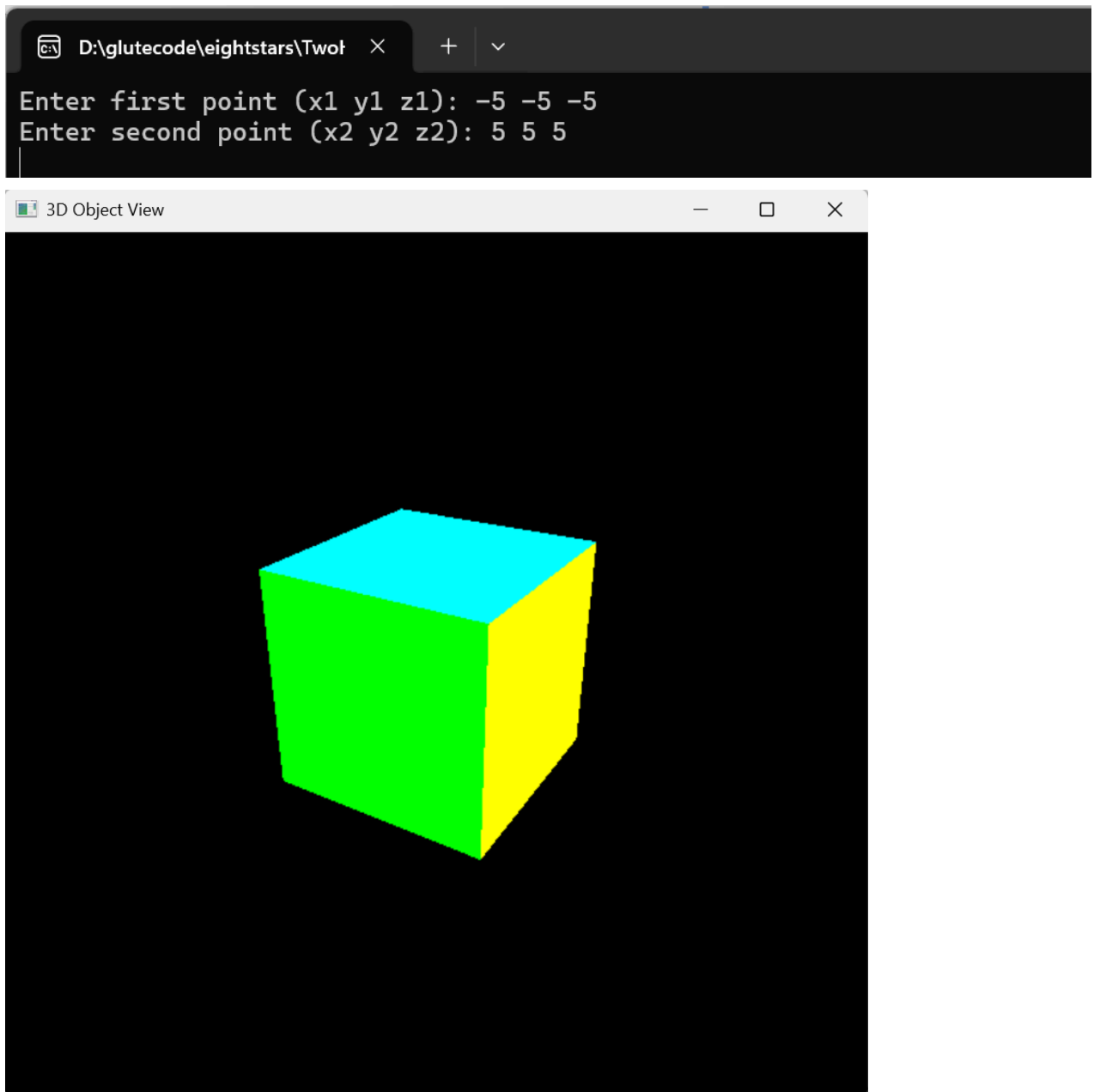
void init()
{
    glEnable(GL_DEPTH_TEST);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(60, 1.0, 1.0, 100.0);
    glMatrixMode(GL_MODELVIEW);
}

int main(int argc, char** argv)
{
    cout << "Enter first point (x1 y1 z1): ";
    cin >> X1 >> Y1 >> Z1;

    cout << "Enter second point (x2 y2 z2): ";
    cin >> X2 >> Y2 >> Z2;

    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(600, 600);
    glutCreateWindow("3D Object View");
    init();
    glutDisplayFunc(display);
    glutMainLoop();

    return 0;
}
```



2. 3D Translation

```
#include <GL/glut.h>
#include <iostream>
using namespace std;

float X1, Y1, Z1;
float X2, Y2, Z2;
```

```
float Tx, Ty, Tz;
float moveX = 0, moveY = 0, moveZ = 0;
void drawCubeFromPoints(float x1, float y1, float z1, float x2, float y2, float z2)
{
    glBegin(GL_QUADS);
    // Front face
    glColor3f(1, 0, 0);
    glVertex3f(x1, y1, z1);
    glVertex3f(x2, y1, z1);
    glVertex3f(x2, y2, z1);
    glVertex3f(x1, y2, z1);
    // Back face
    glColor3f(0, 1, 0);
    glVertex3f(x1, y1, z2);
    glVertex3f(x1, y2, z2);
    glVertex3f(x2, y2, z2);
    glVertex3f(x2, y1, z2);
    // Left face
    glColor3f(0, 0, 1);
    glVertex3f(x1, y1, z1);
    glVertex3f(x1, y1, z2);
    glVertex3f(x1, y2, z2);
    glVertex3f(x1, y2, z1);
    // Right face
    glColor3f(1, 1, 0);
    glVertex3f(x2, y1, z1);
    glVertex3f(x2, y1, z2);
    glVertex3f(x2, y2, z2);
    glVertex3f(x2, y2, z1);
    // Top face
    glColor3f(0, 1, 1);
    glVertex3f(x1, y2, z1);
```

```
        glVertex3f(x1, y2, z2);
        glVertex3f(x2, y2, z2);
        glVertex3f(x2, y2, z1);
        // Bottom face
        glColor3f(1, 0, 1);
        glVertex3f(x1, y1, z1);
        glVertex3f(x2, y1, z1);
        glVertex3f(x2, y1, z2);
        glVertex3f(x1, y1, z2);
        glEnd();
    }
void display()
{
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();
    gluLookAt(15, 15, 25, 0, 0, 0, 0, 1, 0);
    glTranslatef(moveX, moveY, moveZ);
    drawCubeFromPoints(X1, Y1, Z1, X2, Y2, Z2);
    glutSwapBuffers();
}
void keyboard(unsigned char key, int x, int y)
{
    switch (key)
    {
        case 'X': moveX += Tx; break; // move +X
        case 'x': moveX -= Tx; break; // move -X

        case 'Y': moveY += Ty; break; // move +Y
        case 'y': moveY -= Ty; break; // move -Y

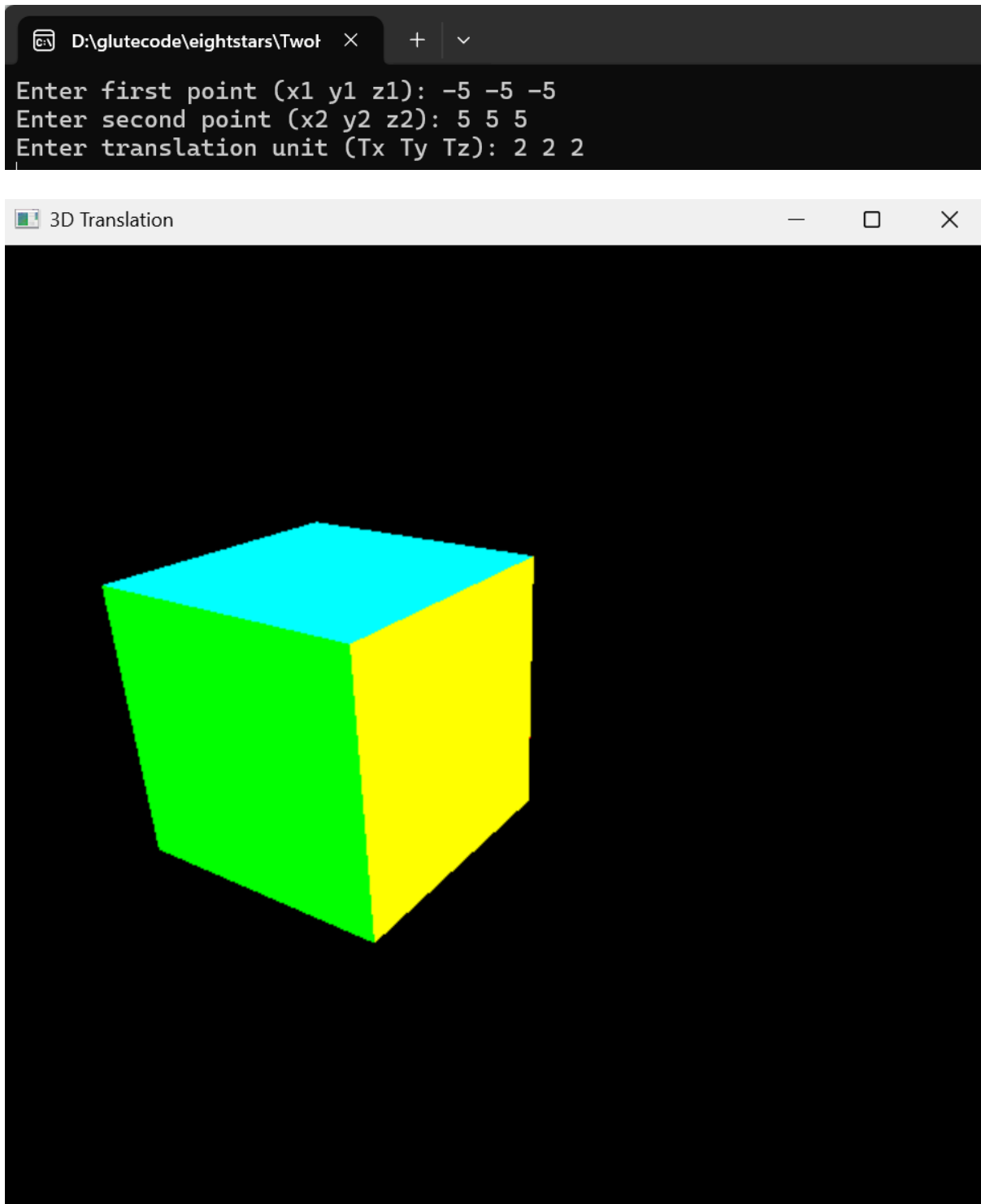
        case 'Z': moveZ += Tz; break; // move +Z
        case 'z': moveZ -= Tz; break; // move -Z
    }
}
```

```
        glutPostRedisplay();
    }

void init()
{
    glEnable(GL_DEPTH_TEST);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(60, 1.0, 1.0, 100.0);
    glMatrixMode(GL_MODELVIEW);
}

int main(int argc, char** argv)
{
    cout << "Enter first point (x1 y1 z1): ";
    cin >> X1 >> Y1 >> Z1;
    cout << "Enter second point (x2 y2 z2): ";
    cin >> X2 >> Y2 >> Z2;

    cout << "Enter translation unit (Tx Ty Tz): ";
    cin >> Tx >> Ty >> Tz;
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(600, 600);
    glutCreateWindow("3D Translation");
    init();
    glutDisplayFunc(display);
    glutKeyboardFunc(keyboard);
    glutMainLoop();
    return 0;
}
```



3. Scaling

```
#include <GL/glut.h>
#include <iostream>
using namespace std;

float X1, Y1, Z1;
float X2, Y2, Z2;
float Sx, Sy, Sz;
float scaleX = 1.0f, scaleY = 1.0f, scaleZ = 1.0f;

void drawCubeFromPoints(float x1, float y1, float z1,
                        float x2, float y2, float z2)
{
    glBegin(GL_QUADS);

    // Front face
    glColor3f(1, 0, 0);
    glVertex3f(x1, y1, z1);
    glVertex3f(x2, y1, z1);
    glVertex3f(x2, y2, z1);
    glVertex3f(x1, y2, z1);
    // Back face
    glColor3f(0, 1, 0);
    glVertex3f(x1, y1, z2);
    glVertex3f(x1, y2, z2);
    glVertex3f(x2, y2, z2);
    glVertex3f(x2, y1, z2);
    // Left face
    glColor3f(0, 0, 1);
    glVertex3f(x1, y1, z1);
    glVertex3f(x1, y1, z2);
    glVertex3f(x1, y2, z2);
    glVertex3f(x1, y2, z1);
    // Right face
    glColor3f(1, 1, 0);
    glVertex3f(x2, y1, z1);
    glVertex3f(x2, y1, z2);
    glVertex3f(x2, y2, z2);
    glVertex3f(x2, y2, z1);
    // Top face
    glColor3f(0, 1, 1);
    glVertex3f(x1, y2, z1);
    glVertex3f(x1, y2, z2);
    glVertex3f(x2, y2, z2);
    glVertex3f(x2, y2, z1);
    // Bottom face
    glColor3f(1, 0, 1);
    glVertex3f(x1, y1, z1);
    glVertex3f(x2, y1, z1);
    glVertex3f(x2, y1, z2);
    glVertex3f(x1, y1, z2);
}
```

```
    glEnd();
}
void display()
{
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();
    gluLookAt(15, 15, 25, 0, 0, 0, 1, 0);
    glScalef(scaleX, scaleY, scaleZ);
    drawCubeFromPoints(X1, Y1, Z1, X2, Y2, Z2);
    glutSwapBuffers();
}
void keyboard(unsigned char key, int x, int y)
{
    switch (key)
    {
        case 'i':
            scaleX += Sx;
            scaleY += Sy;
            scaleZ += Sz;
            break;
        case 'k':
            scaleX -= Sx;
            scaleY -= Sy;
            scaleZ -= Sz;
            if(scaleX < 0.1f) scaleX = 0.1f;
            if(scaleY < 0.1f) scaleY = 0.1f;
            if(scaleZ < 0.1f) scaleZ = 0.1f;
            break;
    }

    glutPostRedisplay();
}

void init()
{
    glEnable(GL_DEPTH_TEST);

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(60, 1.0, 1.0, 100.0);
    glMatrixMode(GL_MODELVIEW);
}

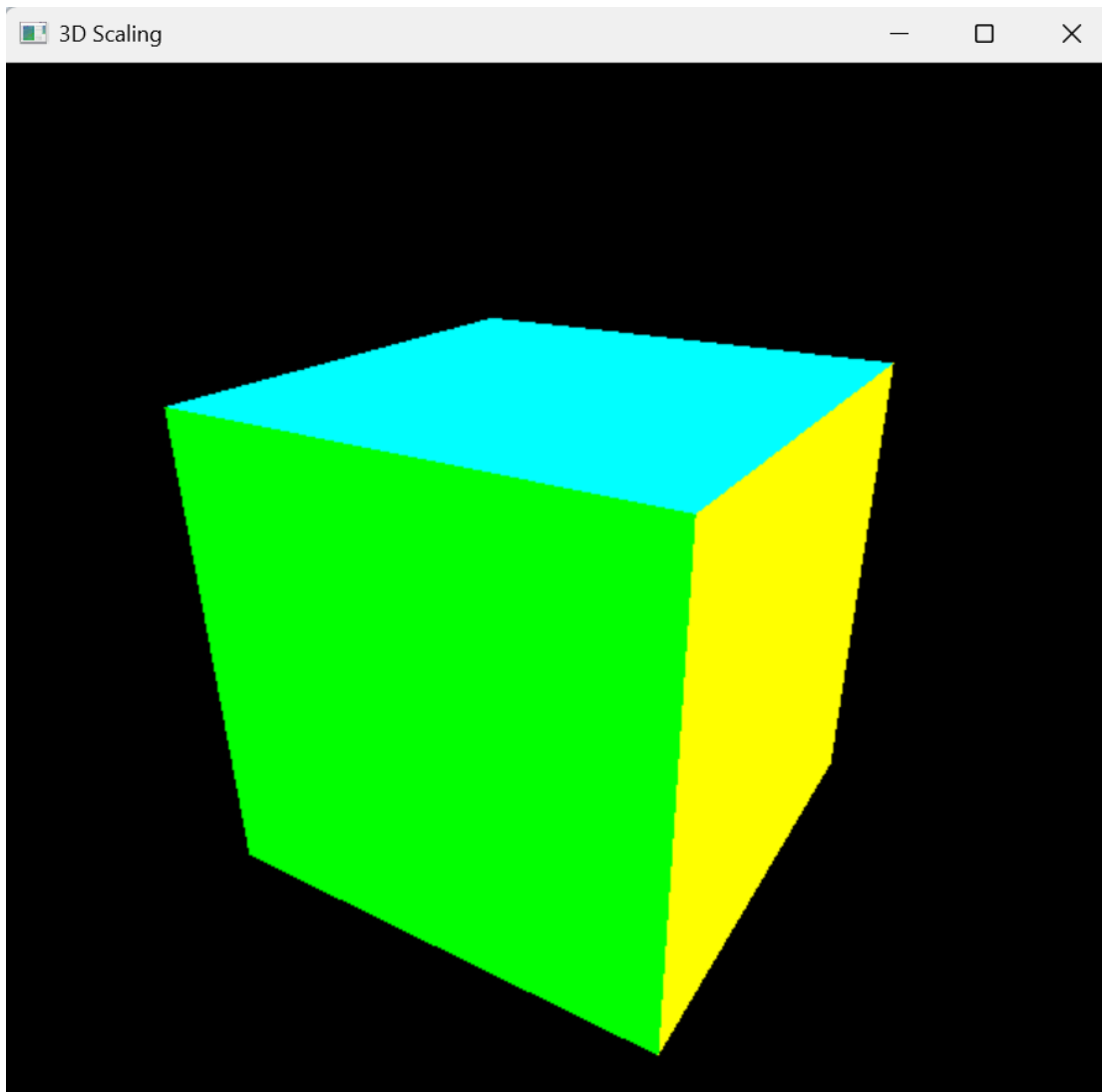
int main(int argc, char** argv)
{
    cout << "Enter first point (x1 y1 z1): ";
    cin >> X1 >> Y1 >> Z1;

    cout << "Enter second point (x2 y2 z2): ";
    cin >> X2 >> Y2 >> Z2;

    cout << "Enter scaling unit (Sx Sy Sz): ";
    cin >> Sx >> Sy >> Sz;
```

```
glutInit(&argc, argv);
glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
glutInitWindowSize(600, 600);
glutCreateWindow("3D Scaling");
init();
glutDisplayFunc(display);
glutKeyboardFunc(keyboard);
glutMainLoop();
return 0;
}
```

```
D:\glutcode\eightstars\TwoT × + ▾
Enter first point (x1 y1 z1): -2 -2 -2
Enter second point (x2 y2 z2): 2 2 2
Enter scaling unit (Sx Sy Sz): 1 1 1
```



4. 3D Rotation

```
#include <GL/glut.h>
#include <iostream>
using namespace std;

float X1, Y1, Z1;
float X2, Y2, Z2;

float angleX = 0.0f;
float angleY = 0.0f;
float angleZ = 0.0f;

float stepX, stepY, stepZ;

void drawCubeFromPoints(float x1, float y1, float z1,
                        float x2, float y2, float z2)
{
    glBegin(GL_QUADS);
    // Front face
    glColor3f(1, 0, 0);
    glVertex3f(x1, y1, z1);
    glVertex3f(x2, y1, z1);
    glVertex3f(x2, y2, z1);
    glVertex3f(x1, y2, z1);
    // Back face
    glColor3f(0, 1, 0);
    glVertex3f(x1, y1, z2);
    glVertex3f(x1, y2, z2);
    glVertex3f(x2, y2, z2);
    glVertex3f(x2, y1, z2);
    // Left face
    glColor3f(0, 0, 1);
    glVertex3f(x1, y1, z1);
    glVertex3f(x1, y1, z2);
    glVertex3f(x1, y2, z2);
    glVertex3f(x1, y2, z1);
    // Right face
    glColor3f(1, 1, 0);
    glVertex3f(x2, y1, z1);
    glVertex3f(x2, y1, z2);
    glVertex3f(x2, y2, z2);
    glVertex3f(x2, y2, z1);
    // Top face
    glColor3f(0, 1, 1);
    glVertex3f(x1, y2, z1);
    glVertex3f(x1, y2, z2);
    glVertex3f(x2, y2, z2);
    glVertex3f(x2, y2, z1);

    // Bottom face
    glColor3f(1, 0, 1);
    glVertex3f(x1, y1, z1);
```

```

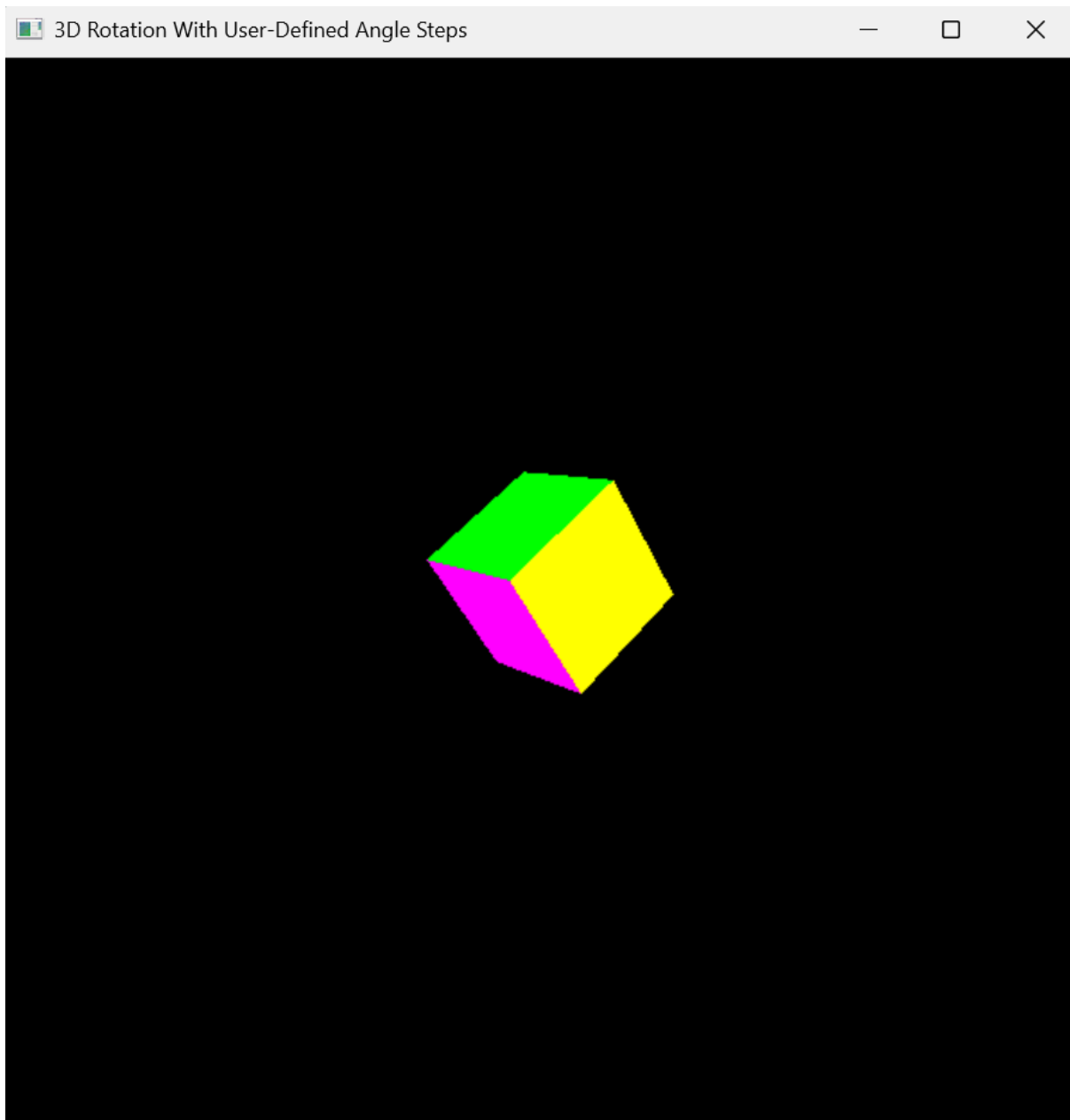
    glVertex3f(x2, y1, z1);
    glVertex3f(x2, y1, z2);
    glVertex3f(x1, y1, z2);
    glEnd();
}
void display()
{
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();
    gluLookAt(15, 15, 25, 0, 0, 0, 1, 0);
    glRotatef(angleX, 1, 0, 0);
    glRotatef(angleY, 0, 1, 0);
    glRotatef(angleZ, 0, 0, 1);
    drawCubeFromPoints(X1, Y1, Z1, X2, Y2, Z2);
    glutSwapBuffers();
}
void keyboard(unsigned char key, int, int)
{
    switch (key)
    {
        case 'X': angleX += stepX; break; //rotate +X
        case 'x': angleX -= stepX; break; //rotate -X
        case 'Y': angleY += stepY; break; //rotate +Y
        case 'y': angleY -= stepY; break; //rotate -Y
        case 'Z': angleZ += stepZ; break; //rotate +Z
        case 'z': angleZ -= stepZ; break; //rotate -Z
    }
    glutPostRedisplay();
}
void init()
{
    glEnable(GL_DEPTH_TEST);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(60, 1.0, 1.0, 100.0);
    glMatrixMode(GL_MODELVIEW);
}
int main(int argc, char** argv)
{
    cout << "Enter first point (x1 y1 z1): ";
    cin >> X1 >> Y1 >> Z1;
    cout << "Enter second point (x2 y2 z2): ";
    cin >> X2 >> Y2 >> Z2;
    cout << "Enter rotation angle steps (stepX stepY stepZ): ";
    cin >> stepX >> stepY >> stepZ;

    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(600, 600);
    glutCreateWindow("3D Rotation With User-Defined Angle Steps");
    init();
    glutDisplayFunc(display);
    glutKeyboardFunc(keyboard);
    glutMainLoop();
}

```

```
return 0;  
}
```

```
D:\glutecode\eightstars\Two1 × + v  
Enter first point (x1 y1 z1): -2 -2 -2  
Enter second point (x2 y2 z2): 3 3 3  
Enter rotation angle steps (stepX stepY stepZ): 5 5 5
```



5. 3D Translation Final

```
#include <GL/glut.h>
#include <iostream>
using namespace std;

float X1, Y1, Z1;
float X2, Y2, Z2;

float angleX = 0.0f;
float angleY = 0.0f;
float angleZ = 0.0f;

float stepX, stepY, stepZ;
float Tx, Ty, Tz;
float Sx, Sy, Sz;

float transX = 0, transY = 0, transZ = 0;
float scaleX = 1.0f, scaleY = 1.0f, scaleZ = 1.0f;

void drawCubeFromPoints(float x1, float y1, float z1,
                        float x2, float y2, float z2)
{
    glBegin(GL_QUADS);

    // Front
    glColor3f(1, 0, 0);
    glVertex3f(x1, y1, z1);
    glVertex3f(x2, y1, z1);
    glVertex3f(x2, y2, z1);
    glVertex3f(x1, y2, z1);

    // Back
```

```
        glColor3f(0, 1, 0);
        glVertex3f(x1, y1, z2);
        glVertex3f(x1, y2, z2);
    glVertex3f(x2, y2, z2);
        glVertex3f(x2, y1, z2);

    // Left
    glColor3f(0, 0, 1);
    glVertex3f(x1, y1, z1); glVertex3f(x1, y1, z2);
    glVertex3f(x1, y2, z2); glVertex3f(x1, y2, z1);

    // Right
    glColor3f(1, 1, 0);
    glVertex3f(x2, y1, z1); glVertex3f(x2, y1, z2);
    glVertex3f(x2, y2, z2); glVertex3f(x2, y2, z1);

    // Top
    glColor3f(0, 1, 1);
    glVertex3f(x1, y2, z1); glVertex3f(x1, y2, z2);
    glVertex3f(x2, y2, z2); glVertex3f(x2, y2, z1);

    // Bottom
    glColor3f(1, 0, 1);
    glVertex3f(x1, y1, z1); glVertex3f(x2, y1, z1);
    glVertex3f(x2, y1, z2); glVertex3f(x1, y1, z2);
        glEnd();
    }
void display()
{
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();
    gluLookAt(15, 15, 25, 0, 0, 0, 0, 1, 0);
    glTranslatef(transX, transY, transZ);
```



```
    glRotatef(angleX, 1, 0, 0);
    glRotatef(angleY, 0, 1, 0);
    glRotatef(angleZ, 0, 0, 1);
    glScalef(scaleX, scaleY, scaleZ);
    drawCubeFromPoints(X1, Y1, Z1, X2, Y2, Z2);
    glutSwapBuffers();
}
void keyboard(unsigned char key, int, int)
{
    switch (key)
    {
        case 'X': angleX += stepX; break; // rotate +X
        case 'x': angleX -= stepX; break; // rotate -X

        case 'Y': angleY += stepY; break; // rotate +Y
        case 'y': angleY -= stepY; break; // rotate -Y

        case 'Z': angleZ += stepZ; break; // rotate +Z
        case 'z': angleZ -= stepZ; break; // rotate -Z

        case 'w': transY += Ty; break; // move +X
        case 's': transY -= Ty; break; // move -X

        case 'd': transX += Tx; break; // move +Y
        case 'a': transX -= Tx; break; // move -Y

        case 'q': transZ += Tz; break; // move +Z
        case 'e': transZ -= Tz; break; // move -Z

        case '+':
            scaleX += Sx; scaleY += Sy; scaleZ += Sz; // scale ++
            break;
        case '-':
```

```
        scaleX -= Sx; scaleY -= Sy; scaleZ -= Sz; // scale --
        if (scaleX < 0.1) scaleX = 0.1;
        if (scaleY < 0.1) scaleY = 0.1;
        if (scaleZ < 0.1) scaleZ = 0.1;
        break;
    }
    glutPostRedisplay();
}

void init()
{
    glEnable(GL_DEPTH_TEST);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(60, 1.0, 1.0, 100);
    glMatrixMode(GL_MODELVIEW);
}

int main(int argc, char** argv)
{
    cout << "Enter first point (x1 y1 z1): ";
    cin >> X1 >> Y1 >> Z1;
    cout << "Enter second point (x2 y2 z2): ";
    cin >> X2 >> Y2 >> Z2;
    cout << "Enter rotation step values (stepX stepY stepZ): ";
    cin >> stepX >> stepY >> stepZ;
    cout << "Enter translation units (Tx Ty Tz): ";
    cin >> Tx >> Ty >> Tz;
    cout << "Enter scaling units (Sx Sy Sz): ";
    cin >> Sx >> Sy >> Sz;

    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(700, 700);
    glutCreateWindow("3D Transformations");
```

```
init();  
glutDisplayFunc(display);  
glutKeyboardFunc(keyboard);  
glutMainLoop();  
return 0;  
}
```

```
D:\glutcode\eightstars\Two1 x + v  
Enter first point (x1 y1 z1): -2 -2 -2  
Enter second point (x2 y2 z2): 2 2 2  
Enter rotation step values (stepX stepY stepZ): 1 1 1  
Enter translation units (Tx Ty Tz): 1 1 1  
Enter scaling units (Sx Sy Sz): 5 5 5
```

3D Transformations

